STOCHASTIC MODELING

MODULE 4 | LESSON 3

THE BCC (1997) MODEL

Reading Time	40 minutes
Prior Knowledge	Heston (1993), Merton (1976), CIR (1985)
Keywords	BCC (1997), Monte-Carlo

In this lesson we are going to put together in one model the three different features we have explored in separate models so far:

- Stochastic volatility (Heston, 1993)
- Jump difussion (Merton, 1976)
- Stochastic interest rates (CIR, 1985)

This is exactly what the Bakshi, Cao, and Chen (1997) model does. Let's explore it!

As usual, let's start with the import the necessary libraries:

In [1]:
```
import numpy as np
from scipy.integrate import quad
```

1. BCC (1997) Model

The Bakshi, Cao, and Chen (BCC) model of 1997 incorporates three very relevant features to model underlying asset prices:

- Stochastic volatility (Heston, 1993)
- Jump diffusion (Merton, 1976)
- Stochastic interests rates (Cox-Ingersoll-Ross, 1985)

At this point, you already know models such as Bates (1996) that incorporate the first two characteristics on the list. Therefore, the transition to BCC (1997) model should be pretty smooth, as it basically only adds a change in the way to set the short-rate as any point t.

The set of SDEs that BCC (1997) relies on is the following:

$$ \ \begin{equation} dS_t = (r_t - r_J) S_t dt + \sqrt{\nu_t} S_t dZ_t^1 + J_t S_t

dN_t \end{equation} $$

$$ \ \begin{equation} d\nu_t = \kappa_\nu (\theta_\nu - \nu_t)dt + \sigma_\nu \sqrt{\nu_t}dZ_t^2 \end{equation} $$

$$ \ \begin{equation} dr_t = \kappa_r (\theta_r - r_t)dt + \sigma_r \sqrt{r_t}dZ_t^3 \end{equation} $$
where Z_t^i are standard Brownian motions, and N_t follows a Poisson process.

As we mentioned before, the first 2 equations are equivalent to a Bates (1996) model with exogenous short rate, r_t. Hence, let's start by recoding this model, so that later on we can compare it to the BCC (1997) estimates.

1.1. Revisiting Bates (1996)

For the purpose of later comparison of Bates (1996) versus BCC (1997) models, we include here the code for Bates that we saw in the previous modules

In [2]:
```
# Call Value function


def B96_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb,
    """
    Valuation of European call option in B96 Model via Lewis (2001)
    Parameters:
    ==========
    S0: float
        initial stock/index level
    K: float
        strike price
    T: float
        time-to-maturity (for t=0)
    r: float
        constant risk-free short rate
    kappa_v: float
        mean-reversion factor
    theta_v: float
        long-run mean of variance
    sigma_v: float
        volatility of variance
    rho: float
        correlation between variance and stock/index level
    v0: float
        initial level of variance
    lamb: float
        jump intensity
    mu: float
        expected jump size
    delta: float
        standard deviation of jump
    ==========
    """
    int_value = quad(
```

```
        lambda u: B96_int_func(
            u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
        ),
        0,
        np.inf,
        limit=250,
    )[0]
    call_value = max(0, S0 - np.exp(-r * T) * np.sqrt(S0 * K) / np.pi * i
    return call_value
```

In [3]:

```
# Heston (1993) characteristic function


def H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    """Valuation of European call option in H93 model via Lewis (2001)
    Fourier-based approach: characteristic function.
    Parameter definitions see function BCC_call_value."""
    c1 = kappa_v * theta_v
    c2 = -np.sqrt(
        (rho * sigma_v * u * 1j - kappa_v) ** 2 - sigma_v**2 * (-u * 1j -
    )
    c3 = (kappa_v - rho * sigma_v * u * 1j + c2) / (
        kappa_v - rho * sigma_v * u * 1j - c2
    )
    H1 = r * u * 1j * T + (c1 / sigma_v**2) * (
        (kappa_v - rho * sigma_v * u * 1j + c2) * T
        - 2 * np.log((1 - c3 * np.exp(c2 * T)) / (1 - c3))
    )
    H2 = (
        (kappa_v - rho * sigma_v * u * 1j + c2)
        / sigma_v**2
        * ((1 - np.exp(c2 * T)) / (1 - c3 * np.exp(c2 * T)))
    )
    char_func_value = np.exp(H1 + H2 * v0)
    return char_func_value
```

In [4]:

```
# Merton (1976) characteristic function


def M76J_char_func(u, T, lamb, mu, delta):
    """
    Adjusted Characteristic function for Merton '76 model: Only jump comp
    """

    omega = -lamb * (np.exp(mu + 0.5 * delta**2) - 1)
    char_func_value = np.exp(
        (1j * u * omega + lamb * (np.exp(1j * u * mu - u**2 * delta**2 *
        * T
    )
    return char_func_value
```

In [5]:

```
# Bates (1996) characteristic function (H93+M76)


def B96_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    """
    Bates (1996) characteristic function
    """
    H93 = H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0)
```

```
    M76J = M76J_char_func(u, T, lamb, mu, delta)
    return H93 * M76J
```

In [6]:
```
# Lewis (2001) integral value of Bates (1996)


def B96_int_func(u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb
    """
    Lewis (2001) integral value for Bates (1996) characteristic function
    """
    char_func_value = B96_char_func(
        u - 1j * 0.5, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    )
    int_func_value = (
        1 / (u**2 + 0.25) * (np.exp(1j * u * np.log(S0 / K)) * char_func_
    )
    return int_func_value
```

1.2. BCC (1997)

Now, the BCC (1997) model will look pretty similar the the Bates (1996), except from where we take the short-rates for discounting. We will, nevertheless, use specific functions, which we will denote with the term BCC for differentiation.

First, we will deal with the 'equity component' which, in essence, is identical to Bates (1996), but leaves room for incorporating interest rates as an input:

In [7]:
```
# Call Value function


def BCC_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb,
    """
    Valuation of European call option in B96 Model via Lewis (2001)
    Parameters:
    ==========
    S0: float
        initial stock/index level
    K: float
        strike price
    T: float
        time-to-maturity (for t=0)
    r: float
        constant risk-free short rate
    kappa_v: float
        mean-reversion factor
    theta_v: float
        long-run mean of variance
    sigma_v: float
        volatility of variance
    rho: float
        correlation between variance and stock/index level
    v0: float
        initial level of variance
    lamb: float
        jump intensity
    mu: float
```

```
        expected jump size
    delta: float
        standard deviation of jump
    ==========
    """
    int_value = quad(
        lambda u: BCC_int_func(
            u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
        ),
        0,
        np.inf,
        limit=250,
    )[0]
    call_value = max(0, S0 - np.exp(-r * T) * np.sqrt(S0 * K) / np.pi * i
    return call_value
```

In [8]:
```
# BCC (1997) characteristic function (H93+M76)


def BCC_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    """
    BCC (1997) characteristic function
    """
    H93 = H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0)
    M76J = M76J_char_func(u, T, lamb, mu, delta)
    return H93 * M76J
```

In [9]:
```
# Lewis (2001) integral value of BCC (1997)


def BCC_int_func(u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb
    """
    Lewis (2001) integral value for BCC (1997) characteristic function
    """
    char_func_value = BCC_char_func(
        u - 1j * 0.5, T, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu,
    )
    int_func_value = (
        1 / (u**2 + 0.25) * (np.exp(1j * u * np.log(S0 / K)) * char_func_
    )
    return int_func_value
```

Next, let's deal with the interest rate component, which is basically a CIR (1985) process. Now, unlike we have done before with the CIR model (first with pure simulation, then with calibration) we now have to build all pricing functions for the CIR (1985) model. Remember that the ZCB prices and their components in CIR were defined as follows:

$$ \ \begin{equation*} B_0(T) = b_1(T) e^{-b_2(T)\mathbf{E_0^Q}(r_t)} \end{equation*} $$
where $$ \ \begin{equation*} b_1(T) = \left[\frac{2\gamma e^{((k_r+\gamma)(T-t)/2)}}{ 2\gamma + (k_r + \gamma)(e^{\gamma (T-t)}-1) } \right]^{\frac{2k_r\theta_r}{\sigma^2_r}} \end{equation*} $$

$$ \ \begin{equation*} b_2(T) = \frac{2(e^{\gamma (T-t)}-1)}{2\gamma + (k_r + \gamma)(e^{\gamma (T-t)}-1)} \end{equation*} $$

with $$ \ \begin{equation*} \mathbf{E_0^Q}(r_t) = \theta_r + e^{-\kappa_r t}(r_0-\theta_r) \end{equation*} $$

So that we can, once we've obtained the ZCB prices, see the implied short-rates as:

$$ \ \begin{equation*} Y(0,T) = - \frac{log(B_0(T))}{T} \end{equation*} $$

Let's then code these functions in:

In [10]:
```
def gamma(kappa_r, sigma_r):
    """
    Gamma function in CIR (1985)
    """
    return np.sqrt(kappa_r**2 + 2 * sigma_r**2)

def b1(alpha):
    """
    b1 function in CIR (1985)
    alpha is the parameter set
    """
    r0, kappa_r, theta_r, sigma_r, t, T = alpha
    g = gamma(kappa_r, sigma_r)
    x = (
        (2 * g * np.exp((kappa_r + g) * (T - t) / 2))
        / (2 * g + (kappa_r + g) * (np.exp(g * (T - t)) - 1))
    ) ** (2 * kappa_r * theta_r / sigma_r**2)

    return x


def b2(alpha):
    """
    b2 function in CIR (1985)
    alpha is the parameter set
    """
    r0, kappa_r, theta_r, sigma_r, t, T = alpha
    g = gamma(kappa_r, sigma_r)
    x = (2 * (np.exp(g * (T - t)) - 1)) / (
        2 * g + (kappa_r + g) * (np.exp(g * (T - t)) - 1)
    )

    return x


def B(alpha):
    """
    ZCB prices in the CIR (1985) model
    """
```

```
    b_1 = b1(alpha)
    b_2 = b2(alpha)
    r0, kappa_r, theta_r, sigma_r, t, T = alpha

    E_rt = theta_r + np.exp(-kappa_r * t) * (r0 - theta_r)

    zcb = b_1 * np.exp(-b_2 * E_rt)

    return zcb
```

So, once we have the full BCC (1997) model pricing functions implemented (including those from CIR), let's do some pricing!

2. Pricing Results and Comparisons

As you should already know by now, in order to properly use these models for pricing, we should first calibrate them to vanilla market prices and interest rates in order to extract the different parameter values that guarantee we are in a risk-neutral setting (and that we actually can apply risk-neutral valuation!).

We will do this in the next lesson. For now, let's just illustrate things with some given parameters:

Short-rate parameters:

- $r_0 = -0.032\%$
- $\kappa_r = 0.068$
- $\theta_r = 0.207$
- $\sigma_r = 0.112$

Stochastic volatility parameters:

- $\kappa_\nu = 18.447$
- $\theta_\nu = 0.026$
- $\sigma_\nu = 0.978$
- $\rho = -0.821$
- $\nu_0 = 0.035$

Jump diffusion parameters:

- $\lambda = 0.008$
- $\mu = -0.600$
- $\delta = 0.001$

In [11]:
```
# Short-rates
r0 = -0.032 / 100
kappa_r = 0.068
theta_r = 0.207
```

```
sigma_r = 0.112

# SV
kappa_v = 18.447
theta_v = 0.026
sigma_v = 0.978
rho = -0.821
v0 = 0.035

# JD
lamb = 0.008
mu = -0.600
delta = 0.001
```

2.1. Results

We will compare the prices obtained under the Bates (1996) and BCC (1997) models for a very simple call option with $S_0=100$, $K=90$, and $T=1$.

Next, we present the results according to the Bates (1996) model, keeping short-rate, r_0 fixed at its initial level:

In [12]:
```
# Option characteristics

S0 = 100
K = 90
T = 1
```

In [13]:
```
B96_call = B96_call_value(
    S0, K, T, r0, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu, delta
)
print("Call value under Bates (1996): ", B96_call)
```

```
Call value under Bates (1996):  12.815339205009323
```

Finally, let's do the same exercise for the same option, under the BCC (1997) model. This requires that, first, we calculate our ZCB prices given CIR model parameters. Then, we will extract the short-rates associated with each maturity and use them for the discounting.

In [14]:
```
B0T = B([r0, kappa_r, theta_r, sigma_r, 0, T])
r = -np.log(B0T) / T
```

Once we have our short-rates for the maturity needed, we can now apply the same call_value function:

In [15]:
```
BCC_call = BCC_call_value(
    100, 90, 1, r, kappa_v, theta_v, sigma_v, rho, v0, lamb, mu, delta
)
print("Call value under BCC (1997): ", BCC_call)
```

```
Call value under BCC (1997):  13.277735226230874
```

Obviously, the only difference between both prices is due to the short-rate used, which, in the case of BCC, should be determined by the calibrated parameters. So it

can very well be the case that the values of these parameters (or not properly obtaining these values) affects and biases our pricing.

3. Conclusion

In this lesson, we have implemented a BCC (1997) model with stochastic volatility, stochastic interest rates, and jump diffusion. We have performed some synthetic pricing exercises, assuming model parameters as given and exogenously determined. In the next lesson, we will perform a full calibration of the BCC (1997) model in order to properly extract the different parameter values necessary for pricing.

References

- Bakshi, Gurdip, et al. "Empirical Performance of Alternative Option Pricing Models." *The Journal of Finance*, vol. 52, no. 5, 1997, pp. 2003-2049.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.